

2.Memory Management

2.1 Define Memory, Types of Memory

2.2 Single contiguous allocation technique

2.3 Multiprogramming concept

2.4 Internal and External fragmentation

2.5 Memory allocation strategies First fit, Best fit, and worse-fit

2.6 Partitioned, Paging, Segmentation. Demand paging techniques.

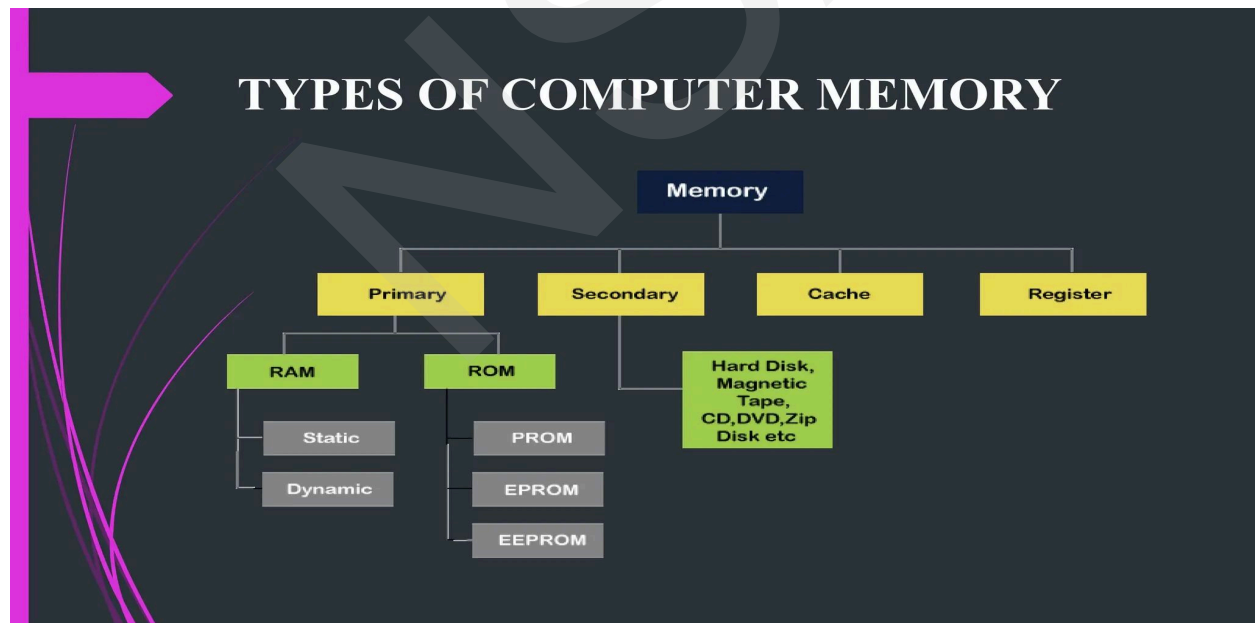
Memory :-

A physical device that stores data or information temporarily or permanently in it is called memory. It's a device where data is stored and processed.

In common, a computer has primary and secondary memories.

Secondary memory stores data and programs for long-term storage or until the time a user wants to keep them in memory, while main memory stores instructions and data during program execution; hence, any program or file that is currently running or executing on a computer is stored in primary memory.

Types of Memory :-



1.Primary Memory (Main Memory)

This is the internal memory of the computer, located physically close to the processor to ensure high-speed performance.

1.Random Access Memory (RAM):

A volatile memory that stores data and programs currently in use. Information is lost when the power is turned off.

1. **SRAM (Static RAM):** Uses transistors to store data; it is extremely fast and expensive, often used for **Cache Memory**.
2. **DRAM (Dynamic RAM):** Uses capacitors and must be refreshed thousands of times per second. It is the standard memory found in most PCs.

2.Read-Only Memory (ROM): A non-volatile memory used to store permanent instructions, such as the computer's boot-up software (BIOS/UEFI).

1. **PROM (Programmable ROM):** Blank chips that can be programmed only once.
2. **EPROM (Erasable Programmable ROM):** Can be erased by exposure to ultraviolet (UV) light and rewritten.
3. **EEPROM (Electrically Erasable Programmable ROM):** Can be erased and rewritten electrically, often used for firmware updates

2. Secondary Memory (Storage)

Also known as auxiliary or external memory, this is used for permanent data storage and is non-volatile.

1. **Magnetic Storage:** Includes Hard Disk Drives (HDD), which use spinning platters to store data magnetically.
2. **Solid-State Storage:** Includes Solid State Drives (SSD) and USB Flash Drives, which use electronic circuits for faster, more durable storage than traditional hard drives.
3. **Optical Storage:** Devices like CDs, DVDs and Blu-ray discs that use laser light to read and write data.

3.Specialized Memory Types

1. **Cache Memory:**
High-speed volatile memory situated between the CPU and RAM to store frequently used data and speed up processing.
2. **Registers:**
Small, ultra-fast storage locations located directly inside the CPU that hold immediate data for calculation.
Extremely fast, small memory located near the CPU, stores frequently used data for instant access, often uses SRAM
3. **Virtual memory:**
A memory management technique that uses a portion of the hard drive (secondary memory) to act as an extension of the system's RAM.

Memory Management in Operating System

Memory management is the process of controlling and organising a computer's memory by allocating portions, called blocks, to different executing programs to improve the overall system performance.

The most important function of an operating system is to manage primary memory.

It Supports multiple processes simultaneously in memory.

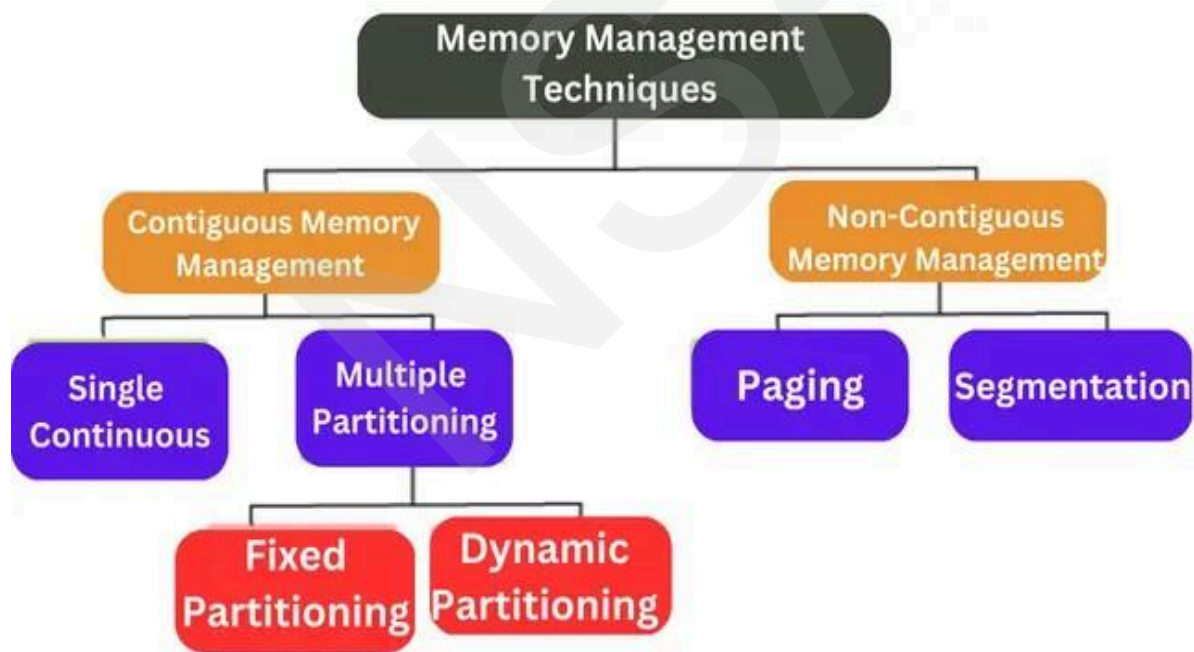
It Protects processes from unauthorized access.

It Enables swapping and virtual memory efficiently.

Techniques in Memory Allocation :-

Used by an operating system to efficiently allocate, utilize, and manage memory resources for processes.

Various techniques help the operating system manage memory effectively. They can be broadly categorized into:



Swapping :-

Swapping is a memory management technique where processes are temporarily moved between main memory and secondary storage to free up memory for other processes.

Allows multiple processes to run efficiently. Lower-priority processes can be swapped out for higher-priority ones. Swapped-out processes resume when loaded back into memory. Transfer time depends on the amount of data moved.

Switching :-

Switching is the process where control is continuously transferring from user mode to kernel mode.

User mode is mode when the program written is running on the system and at that time when input output interrupt occurs then control is transferred from user to kernel mode.

This process is happening continuously in memory.

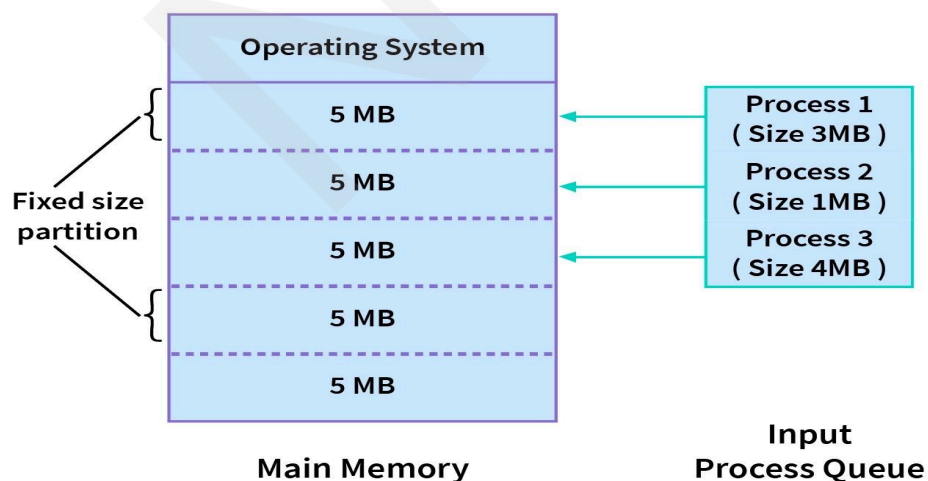
Contiguous Memory Allocation :-

Contiguous memory allocation is a memory management technique used by operating systems to allocate a single continuous block of memory to a process.

When a process is loaded into memory for execution, the operating system (OS) assigns a contiguous block that meets its size requirements.

This method ensures efficient access to memory because all the allocated memory for a process is placed in one sequential block.

However, it comes with limitations such as fragmentation and restricted flexibility, which can impact overall system performance.



SCALER
Topics

Fig. CONTIGUOUS MEMORY ALLOCATION

Characteristics of Contiguous Memory Allocation :-

1. Adjacent Memory Blocks :-

In contiguous memory allocation, a process is assigned a single continuous block of memory in the system's RAM. This means that all memory allocated to a process is placed sequentially, ensuring faster access and efficient CPU caching.

2. Fixed or Variable Partitioning :-

Contiguous memory can be allocated using either fixed-size partitioning or variable-size partitioning.

- **Fixed-size partitioning:**

The memory is divided into equal-sized partitions before processes are loaded. If a process is smaller than a partition, the remaining space is wasted (internal fragmentation).

- **Variable-size partitioning:**

The OS dynamically assigns memory blocks based on the size of the processes. This reduces internal fragmentation but introduces external fragmentation, where free spaces between allocated blocks make it difficult to assign memory efficiently.

3. Prone to Fragmentation :-

Contiguous memory allocation suffers from two types of fragmentation:

- **Internal Fragmentation :**

This occurs when a process does not use up all the memory in its assigned partition, leading to wasted space within the allocated block.

- **External Fragmentation :**

This happens when free memory is scattered in small, non-contiguous blocks, making it difficult to allocate large processes. Over time, this reduces system efficiency and may require memory compaction to merge free spaces.

4. Efficient Address Translation :-

Since the entire process is stored in a single contiguous block, the address translation process is simple. The logical address can be converted into a physical address using a base and offset mechanism. This reduces the overhead on the CPU and ensures quick access to instructions and data.

5. Limited Flexibility :-

Once a process is assigned a memory block, it cannot expand or shrink dynamically. If a process needs more memory than allocated, it has to be relocated to a larger block, which can be time-consuming. This makes it inefficient for handling processes with varying memory requirements.

Advantages and Disadvantages of Contiguous Memory Allocation

Contiguous memory allocation is a memory management technique where processes are assigned a single continuous block of memory. While this method offers simplicity and efficiency, it also has limitations such as fragmentation and process size restrictions.

Advantages of Contiguous Memory Allocation :-

1. Efficient Memory Access

Since all memory blocks assigned to a process are adjacent, the CPU can access data quickly and sequentially.

This leads to faster read and write operations, improving overall system performance.

2. Simple Memory Management

Tracking allocated and free memory blocks is straightforward in contiguous memory allocation.

Since each process gets a single continuous block, the operating system (OS) only needs to store the starting address and size of each allocation.

3. Reduced Overhead

Contiguous allocation requires less computational overhead than non-contiguous methods, where memory is allocated in scattered locations.

That's why contiguous allocation avoids extra processing steps.

4. Predictable Performance and Better Caching

Since memory is allocated in a single block, fewer memory accesses are required. This enhances cache efficiency, as the processor can prefetch sequential instructions and data into the cache.

Disadvantages of Contiguous Memory Allocation :-

1. External Fragmentation

Over time, as processes terminate and new ones are allocated, small unused memory gaps (fragments) appear between allocated blocks. These gaps cannot always be used efficiently, leading to wasted memory.

Example: If a 10MB process is removed from memory, leaving a 10MB gap, and a new process requiring 12MB arrives, it cannot fit, despite available free space.

2. Internal Fragmentation (Memory Waste in Fixed Partitioning)

In fixed-size partitioning, memory partitions are created before execution. If a process does not fully utilize its assigned partition, the remaining memory is wasted.

Example: If a 5MB process is allocated to a 10MB partition, 5MB remains unused, leading to internal fragmentation.

3. Limited Process Size

In fixed partitioning, processes must fit within predefined partition sizes, restricting the execution of large processes. Even in variable partitioning, memory allocation depends on the availability of large contiguous spaces.

Impact: Large processes may not execute due to insufficient contiguous memory, even if the total free memory is sufficient.

4. Complex Allocation Strategies in Variable Partitioning

While variable partitioning allows flexible memory allocation, it increases runtime complexity due to fragmentation management and dynamic allocation.

Impact: The OS must use allocation strategies (First-Fit, Best-Fit, Worst-Fit), increasing computational overhead.

Types of Partitioned Memory Allocation :-

Contiguous memory allocation is categorized into two main types based on how memory is partitioned and allocated to processes.

These are Fixed-Size Partitioning and Variable-Size Partitioning.

Each method has its own advantages and drawbacks that affect system performance and memory utilization.

1. Fixed-Size Partitioning

Fixed-size partitioning, also known as static partitioning, is a memory allocation technique in which RAM is divided into equal-sized partitions before program execution.

Each partition can hold only one process at a time, regardless of the process size.

Example

Consider a system with 1GB of RAM, which is divided into 100 partitions of 10MB each. When a process enters the system, it is allocated one of these partitions. Even if the process requires less than 10MB, it still occupies the entire partition, leading to wastage of memory.

Advantages of Fixed-Size Partitioning :-

1. Simple and Easy to Implement

Since partition sizes are predetermined, the operating system (OS) does not need to dynamically allocate memory, making it easy to manage.

2. Fast Allocation and Deallocation

Because partitions are fixed, the OS does not need to search for a suitable block, reducing the time required for memory allocation and deallocation.

3. Minimal Overhead in Tracking Allocations

Memory management is straightforward as the OS only needs to track whether a partition is occupied or free.

Disadvantages of Fixed-Size Partitioning :-

1. Internal Fragmentation

If a process does not fully utilize its allocated partition, the remaining space is wasted. For example, a 5MB process in a 10MB partition results in 5MB of unused memory, leading to inefficient memory utilization.

2. Fixed Size Limitation

If a process requires 15MB of memory, it cannot fit into a 10MB partition, even if multiple partitions are free. This limitation restricts the execution of larger processes.

3. Inefficient for Dynamic Workloads

Systems running processes of varying sizes may struggle with this method, as processes must conform to predefined partition sizes, limiting flexibility.

2. Variable-Size Partitioning (Dynamic Partitioning)

Variable-size partitioning, also known as dynamic partitioning, is a memory allocation technique in which RAM is allocated dynamically based on the exact size of the process. Unlike fixed partitioning, no predefined partitions exist, and memory blocks are assigned as needed.

Example

If a process requires 3MB of memory, the OS allocates exactly 3MB, leaving the remaining memory available for other processes. This approach maximizes memory utilization and reduces internal fragmentation.

Advantages of Variable-Size Partitioning :-

1. No Internal Fragmentation

Since each process receives only the memory it requires, there is no unused space within allocated partitions, leading to efficient memory utilization.

2. More Efficient Use of Memory

Larger processes can be allocated space without restrictions, as there are no fixed partition

sizes. A 15MB process can be allocated 15MB of memory, instead of being forced into a smaller partition.

3. Greater Flexibility

The system can handle processes of varying sizes dynamically, making it ideal for multitasking environments and modern computing needs.

Disadvantages of Variable-Size Partitioning :-

1. External Fragmentation

As processes terminate and new ones are loaded, small gaps are created in memory. Over time, these gaps prevent large processes from being allocated, leading to inefficient memory utilization.

2. Complex Memory Management

The OS must continuously track free memory blocks and use allocation strategies such as First Fit, Best Fit, or Worst Fit to manage available space efficiently. This increases computational overhead.

Fixed-Size Vs Variable-Size Partitioning

Feature	Fixed-Size Partitioning	Variable-Size Partitioning
Internal Fragmentation	Yes (unused space within partitions)	No (allocates exact size)
External Fragmentation	No	Yes (free memory gets fragmented)

Flexibility	Low (predefined partition sizes)	High (adjusts to process size)
Memory Utilization	Inefficient	More efficient
Complexity	Low (easy to manage)	High (requires tracking memory dynamically)
Best Suited For	Systems with predictable workloads	Systems with dynamic workloads

Memory Allocation Strategies

Efficient memory allocation is crucial for optimizing system performance and resource utilization. When a process requests memory, the operating system (OS) must decide how to allocate an available block. There are three primary strategies used for this purpose:

1. First-Fit
2. Best-Fit
3. Worst-Fit

Each of these strategies has its own advantages and disadvantages, impacting speed, fragmentation, and memory efficiency.

1. First-Fit Memory Allocation :-

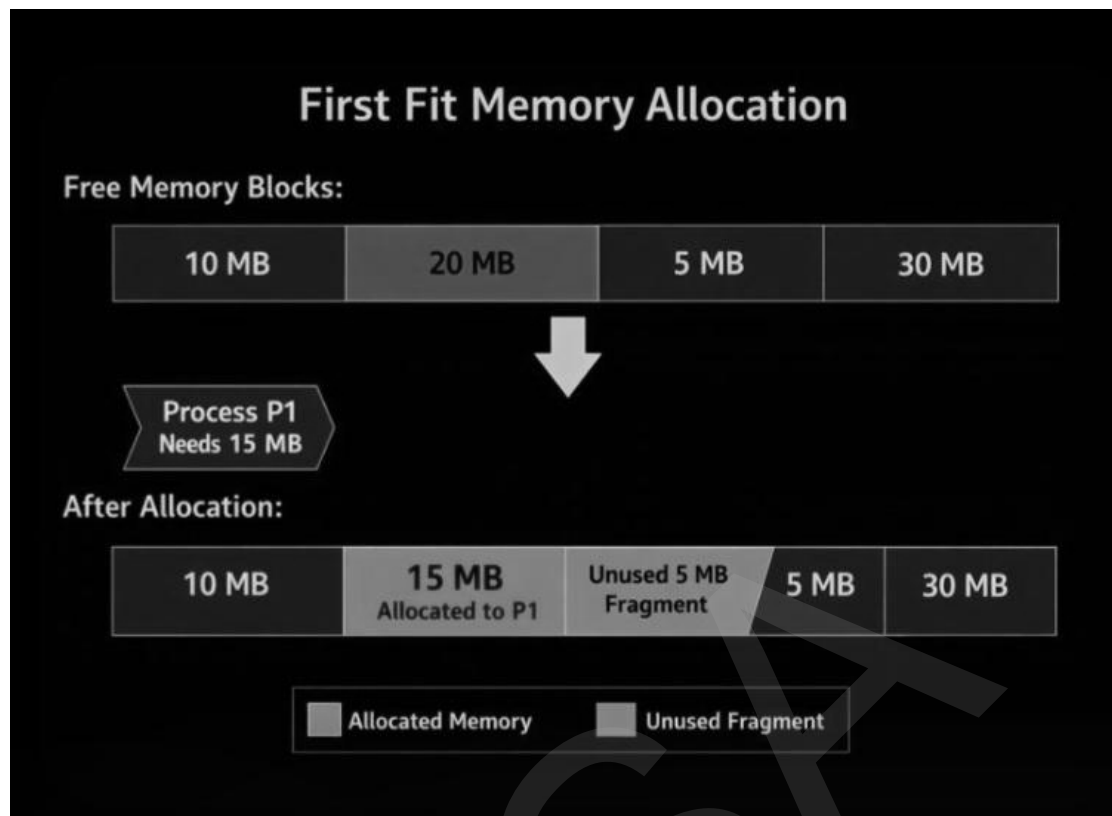
First-Fit is a simple memory allocation strategy that assigns the **first available memory block** that is large enough to place the process.

The OS scans the list of free memory blocks from the beginning and selects the first suitable block.

If the allocated block is larger than the required space, the remaining memory is left as a free block for future allocations.

Example

Consider a memory with free blocks of 10MB, 20MB, 5MB, and 30 MB. If a process requires 15MB, First-Fit will allocate the 20MB block, leaving a 5MB fragment as unused memory.



Advantages of First-Fit :-

1. Fast and Simple Implementation

Since the OS selects the first suitable block it finds, allocation is quick and requires minimal processing.

2. Efficient for Large Numbers of Requests

First-Fit performs well in systems with a high number of memory requests, as it does not scan the entire memory space unnecessarily.

Disadvantages of First-Fit :-

1. Prone to External Fragmentation

Since memory is allocated from the first suitable block, small unused gaps are left behind. Over time, these gaps accumulate and make it difficult to allocate larger processes.

2. Uneven Memory Utilization

Memory blocks at the beginning of the list get allocated frequently, while blocks at the end remain unused, leading to inefficient memory distribution.

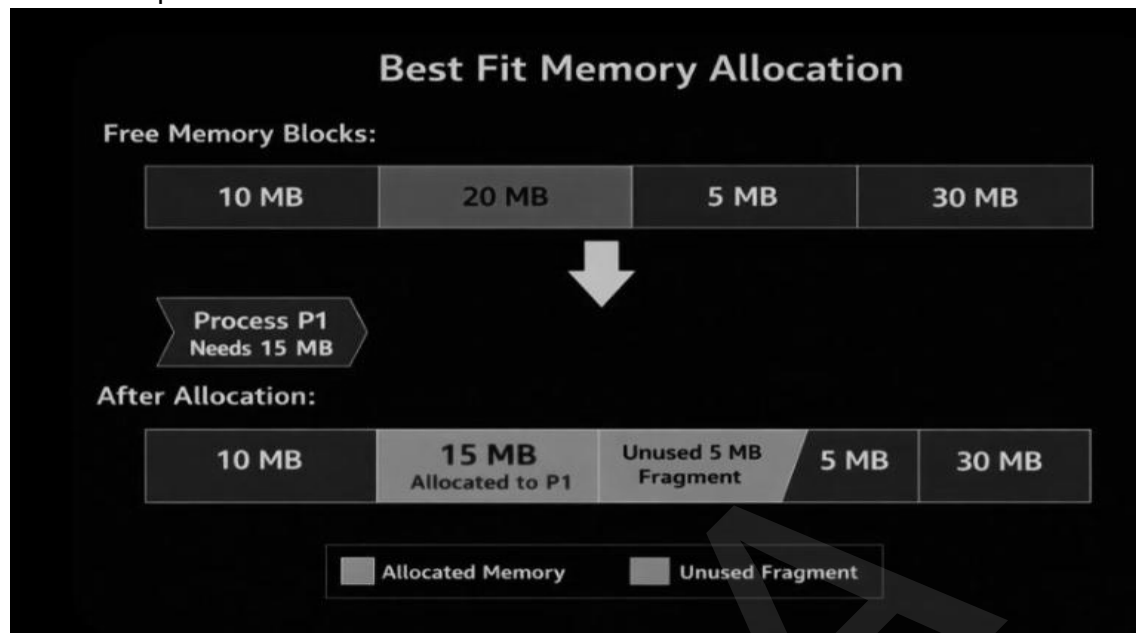
2. Best-Fit Memory Allocation

Best-Fit searches for the smallest available memory block that can accommodate the process. This ensures minimal wasted space, as the process fits as tightly as possible into the block.

Example

Free memory blocks are 10MB, 20MB, 5MB, and 30MB and a process requires 15MB. Now there are two possibilities for allocation of memory blocks that are 20MB and 30MB.

Best-Fit will allocate the 20MB block (instead of 30MB) because it is the smallest available block that fits the process.



Advantages of Best-Fit :-

1. Minimizes Wasted Memory

By choosing the smallest suitable block, Best-Fit reduces internal fragmentation, leading to more efficient memory usage.

2. Better for Memory-Constrained Systems

Since this strategy conserves memory, it is useful in environments with limited resources.

Disadvantages of Best-Fit :-

1. Slower Allocation Time

The OS must scan the entire list of free memory blocks to find the most suitable one, increasing processing time.

2. Can Lead to More Fragmentation Over Time

Because small blocks are allocated precisely, it can result in tiny, unusable gaps in memory, which may become difficult to manage.

3. Worst-Fit Memory Allocation :-

Worst-Fit is a memory allocation technique in which the largest available free memory block is allocated to the process.

The idea is to leave a large remaining fragment that can be used by future processes.

Worst-Fit allocates the largest available memory block, ensuring that the leftover space is still large enough for future allocations.

Example

If free memory blocks are 10MB, 20MB, 5MB, and 30MB, and a process requires 15MB, Worst-Fit will allocate the 30MB block, leaving behind a 15MB free space.

Advantages of Worst-Fit :-

1. Keeps Larger Memory Blocks Available

By always allocating from the largest available block, Worst-Fit ensures that future large processes can still be accommodated.

2. Reduces Frequent Fragmentation in Some Cases

Since it leaves behind larger free memory blocks, fragmentation may be less severe compared to Best-Fit in some scenarios.

Disadvantages of Worst-Fit :-

1. Inefficient Memory Utilization

Allocating from the largest block may lead to more wasted space, as the remaining part might be too small for future processes.

2. Not Suitable for Small Processes

Smaller processes may struggle to fit efficiently into the remaining space, leading to unallocated memory gaps.

Fragmentation in Contiguous Memory Allocation :-

Fragmentation is a major issue in contiguous memory allocation, where processes are assigned a single continuous block of memory.

Over time, as processes are allocated and deallocated, memory becomes inefficiently utilized, leading to wasted space that cannot be used effectively.

Fragmentation can be classified into two main types:

- **External Fragmentation** – Occurs due to small free memory gaps between allocated blocks.
- **Internal Fragmentation** – Occurs when allocated memory is larger than required, leaving unused space within partitions.

Both forms of fragmentation lead to poor memory utilization and can affect the performance and efficiency of a system.

1. External Fragmentation

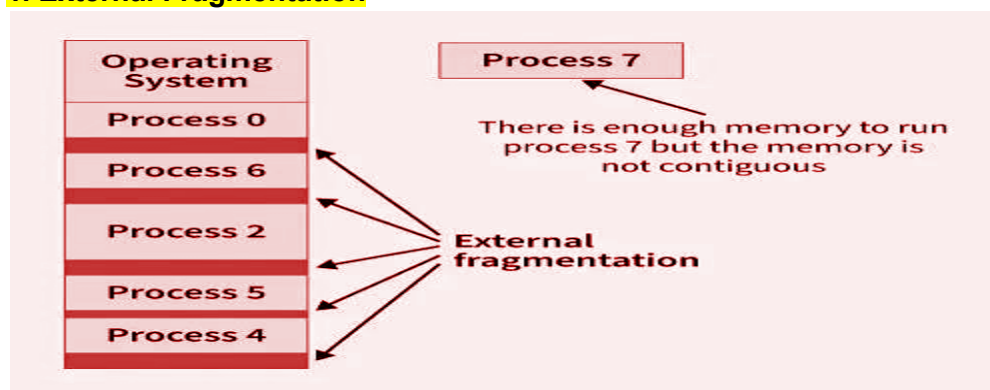


Fig. External Fragmentation

External fragmentation occurs when free memory exists, but it is scattered in small non-contiguous blocks, preventing the allocation of large processes. Even if the total free memory is sufficient, a process may fail to execute because no single continuous block is available.

Cause of External Fragmentation

- When processes terminate, their allocated memory is released.
- This creates gaps between occupied memory regions.
- Over time, these gaps accumulate, making it difficult to allocate large processes, even though there is enough total free memory.

2. Internal Fragmentation

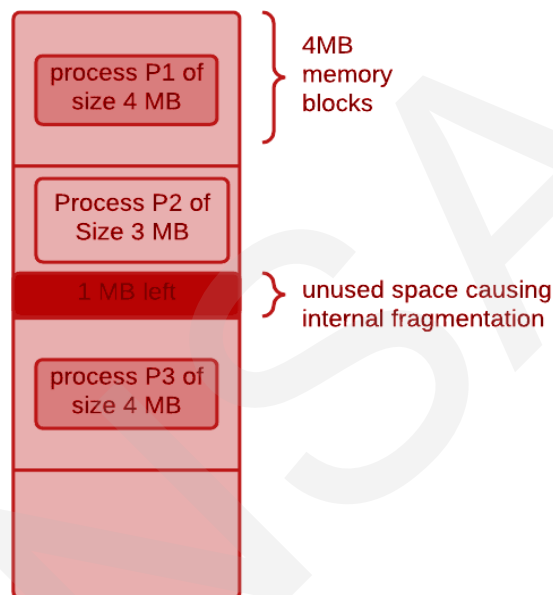


Fig. Internal Fragmentation

Internal fragmentation occurs when fixed-size partitions are used, and a process does not fully utilize its allocated memory. The remaining unused memory within the allocated partition is wasted.

Cause of Internal Fragmentation

- Fixed partitioning assigns memory in predefined blocks.
- If a process is smaller than its allocated partition, the extra space remains unused.
- This leads to inefficient memory utilization.

Multiprogramming concept :-

A multiprogramming operating system allows multiple programs or tasks or jobs to present in main memory at one time and executes more than one program by using a single CPU.

It maximizes efficiency by switching between tasks when one is waiting for I/O, preventing the CPU from becoming idle, and creating the illusion of simultaneous execution for better resource use and throughput.

It manages resources like memory and I/O, while the OS performs context switching to move between processes, keeping the CPU busy.

How it Works

1. Multiple Programs in Memory:

Several jobs (processes) are loaded into main memory at once.

2. CPU Allocation:

- The OS could pick and start the execution of one of the jobs in memory,
- whenever the currently running job does not need CPU and it needs to wait for I/O that means the job is working with I/O
- at that time the OS quickly switches the CPU to another job in main memory that is ready to run
- and the CPU executes a portion of it till the job issues a request for I/O and so on.

Example :-

Let's P1 and P2 are two programs present in the main memory.

The OS picks one program and starts executing it.

During execution if the P1 program requires I/O operation, then the OS will simply switch to the P2 program.

If the p2 program requires I/O then again it switches to P3 and so on.

If there is no other program remaining after P3 then the CPU will pass its control back to the previous program.

Advantages :-

1. Better CPU Utilization: the CPU never goes to idle state.
2. Increased Throughput: More jobs get completed in a given time.
3. Efficient Resource Use: Keeps system resources busy.
4. È utilization is efficient.

Disadvantages :-

1. CPU scheduling is compulsory because lots of jobs are ready to run on CPU simultaneously.
2. The user is not able to interact with jobs when it is executing.
3. Programmers also cannot modify a program that is being executed.

Non-Contiguous Memory Allocation :-

Memory management technique in which a process is divided into smaller parts and these parts are stored in different, non-adjacent locations in main memory.

Unlike contiguous allocation, the entire process does not need to be placed in a single continuous block of memory.

This technique is widely used in modern operating systems because it improves memory utilization and reduces fragmentation problems.

Features of Non-Contiguous Memory Allocation :-

1. A process can be stored in multiple memory locations
2. Improves utilization of available memory
3. Reduces external fragmentation
4. Requires address translation using hardware support (MMU)

Advantages :-

1. Better memory utilization
2. Supports large programs
3. Eliminates the need for contiguous free memory

Disadvantages:-

1. More complex than contiguous allocation
2. Additional overhead for address translation
3. Requires extra memory for tables (page table / segment table)

Techniques Used in Non-Contiguous Memory Allocation :-

Paging :-

In the paging process is divided into blocks called pages and memory is divided into same size blocks called frames.

Paging is the process of moving parts of a program or process called pages, from secondary storage into the main memory (RAM).

The main idea behind paging is to break a program into smaller fixed-size blocks called pages.

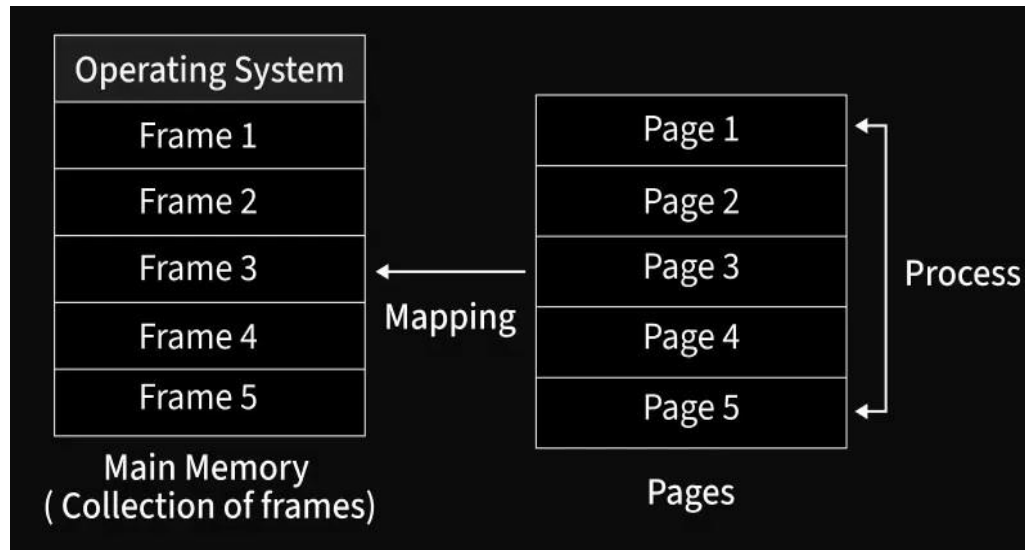
- The process does not have to be allocated in a contiguous memory space.
- The whole process does not have to be in main memory, some pages can be present and some pages can be loaded when needed.
- It allows more processes and even processes larger than main memory to run.
- Memory allocation is simplified as memory is always allocated in fixed sized pages.

Advantages of Paging :

1. No external fragmentation
2. Efficient memory utilization
3. Easy to allocate memory

Disadvantages of Paging :

1. Internal fragmentation
2. Extra memory needed for page table
3. Slower due to address translation



To keep track of where each page is stored in memory, the operating system uses a page table. This table shows the connection between the logical page numbers and the physical page frames (actual locations in RAM).

Note: The memory management unit uses the page table to convert logical addresses into physical addresses, so the program can access the correct data in memory.

Segmentation :-

Divides a process into logical segments of variable size such as code, data, and stack
Segmentation = divide program by LOGIC, not by size

In **Segmentation**:

- Program is divided into logical parts called segments
- Each segment has variable size
- Segments are stored anywhere in RAM

Advantages of Segmentation

1. Matches program structure
2. Easy sharing (code segment)
3. Better protection

Disadvantages of Segmentation

1. External fragmentation
2. Complex memory management

Examples of segments:

- Code segment
- Data segment
- Stack segment
- Heap segment

Real-life example :-

Think of a program like a notebook:

Chapter 1 → Introduction

Chapter 2 → Theory

Chapter 3 → Programs

Chapter 4 → Appendix

Each chapter has a different size but meaningful.

- These chapters are Segments.

Address format :-

Logical Address in Segmentation :- (Segment Number, Offset)

Segment number → which segment

Offset → position inside that segment

Segment Table :-

OS keeps a Segment Table with:

Base → starting address of segment in RAM

Limit → size of segment

Segment	Base	Limit
0 (Code)	1000	400
1 (Data)	2000	300
2 (Stack)	3000	200

Address Calculation (step-by-step)

Given:

Logical address = (1, 150)

Step 1 : Check limit

Segment 1 limit = 300

Since:

$150 < 300$

Address is valid.

Step 2 : Calculate Physical Address

Physical Address = Base + Offset

= $2000 + 150$

= 2150

Done.

If offset is greater than limit :-

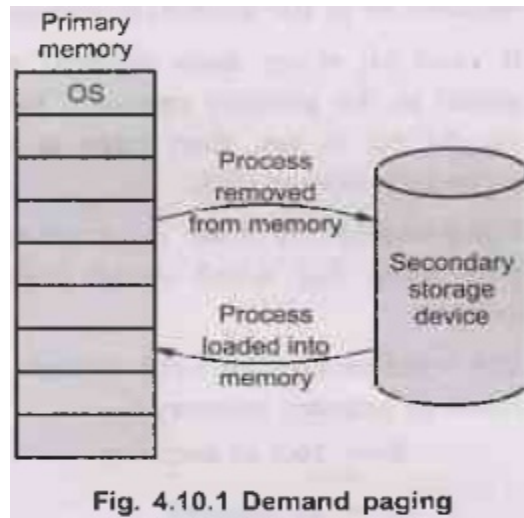
Offset $\geq$ Limit → Segmentation fault

OS will stop the program.

Demand Paging :-

Demand Paging loads only the pages a process actually needs into memory. Reduces unnecessary memory usage and I/O.

- It loads pages from disk into main memory only when they are needed by a program.
- This approach saves memory space by keeping only the required pages in memory, reducing memory allocation costs and improving memory use.
- However, the initial access time for pages not in memory can delay program execution.



In demand paging, the pages of a process are loaded into memory only when they are needed (i.e., on demand) instead of loading all pages at once when the process starts. To understand demand paging, suppose that a process is trying to access a page that is not currently in physical memory (RAM).

It will lead to a **page fault**.

So, the operating system loads the required pages from the virtual memory (disk) into physical memory.

This happens on demand of process, hence the name **demand paging**.

If the frames of the physical memory are full during the loading of new pages, then the OS uses **page replacement algorithms** to decide which page needs to be removed to make space for the new page.